



Master of Engineering

ENGE997 Research Project

AI-Enabled Drone Surveillance for Suspicious Activity Detection Using YOLO and Graph Neural Networks

School of Engineering

Faculty of Design and Creative Technologies

Auckland University of Technology

Academic Supervisor: Dr. Peter Han Joo Chong

Student Name: Shivani Dhandabani

Student ID: 25277528

Year of Submission: 2026

A dissertation submitted to the Auckland University of Technology in partial fulfillment of the requirements for the degree of Master of Engineering

Acknowledgement

I would like to express my sincere appreciation to my academic supervisor, Dr. Peter Han Joo Chong, for providing guidance, technical advice, and encouragement throughout the development of this research project. The project required knowledge from computer vision, artificial intelligence, graph modelling, and software implementation, and the supervisory feedback helped refine the project direction from simple object detection toward a complete surveillance event analysis pipeline.

I also acknowledge the support of the Auckland University of Technology and the School of Engineering for providing the academic environment in which this work was conducted. I am grateful for the availability of open-source tools and research datasets, including Ultralytics YOLO, OpenCV, PyTorch, SQLite, VisDrone, and related aerial video resources, which enabled practical implementation and evaluation of the proposed system.

Finally, I would like to thank my family and friends for their patience, motivation, and encouragement during my postgraduate studies. Their support helped me remain focused during the challenging stages of coding, testing, debugging, and report writing.

Declaration of Originality

I, Shivani Dhandabani, hereby declare that the work presented in this report is entirely my own, except as acknowledged in the text or references. I also declare that this work has not been submitted previously, either in whole or in part, for a degree at this or any other institution.

Name: Shivani Dhandabani

Date: 26th May 2026

Attestation of Authorship

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person, except where explicitly defined in the acknowledgements, nor material which to a substantial extent has been submitted for the award of any other degree or diploma of a university or other institution of higher learning.

Abstract

Recent advances in Artificial Intelligence (AI), computer vision, and drone technologies have transformed surveillance applications across public safety, smart city management, traffic monitoring, and critical infrastructure protection. Traditional surveillance systems rely heavily on fixed cameras and human observation, creating challenges such as operator fatigue, delayed response, inconsistent judgment, and reduced efficiency when monitoring large-scale environments. Drone surveillance platforms provide flexible aerial coverage and enable monitoring of wide geographic areas; however, extracting meaningful behavioral information from drone footage remains a complex engineering problem due to moving viewpoints, object scale variation, occlusion, lighting changes, and computational constraints.

This research proposes an AI-powered drone surveillance system capable of automatically detecting suspicious activities through object detection, tracking, behavioral analysis, graph-based reasoning, and structured event management. The proposed framework integrates YOLOv8 object detection, multi-object tracking, Graph Neural Networks (GNN), SQL database management, and rule-based behavioral logic within a unified surveillance architecture. The system processes uploaded drone or surveillance videos frame-by-frame using computer vision techniques. Objects such as pedestrians, motorcycles, scooters, cars, and other vehicles are detected using a YOLO-based model. Detected objects are assigned tracking identities so that movement history and object persistence can be analyzed over time.

The suspicious behavior detection component focuses primarily on loitering and vehicle gathering. Loitering is identified when a tracked person remains within a predefined spatial region for longer than a selected time threshold while demonstrating limited movement. Vehicle gathering is detected when a group of vehicles remains visible or clustered for a sustained period. To reduce false alerts, the system considers temporal persistence rather than relying on single-frame detections. SQL database tables are

used to record suspicious events, including event type, track identity, start time, end time, duration, final status, action taken, and human-readable summaries. This provides structured evidence for reviewing and auditing detected events after video processing.

Graph Neural Networks are introduced to improve contextual understanding of interactions between detected entities. Unlike traditional object detection systems that analyze objects independently, graph-based modelling represents people and vehicles as nodes and spatial relationships as edges. This enables the system to reason about group behavior and suspicious interaction patterns. The implemented prototype demonstrates how detection, tracking, behavioral rules, graph reasoning, and SQL event logging can be combined to produce a more intelligent surveillance pipeline.

Experimental testing using surveillance-style videos showed that the system could generate annotated output videos, detect vehicle gathering events, identify loitering scenarios, and store resolved or active suspicious events in SQL tables. The main limitations include processing time, threshold sensitivity, tracking errors, limited dataset size, and reduced robustness under challenging environmental conditions. Overall, the project demonstrates a practical AI-enabled surveillance framework that extends object detection into behavior-aware suspicious event analysis.

Keywords: Drone Surveillance, Artificial Intelligence, YOLOv8, Graph Neural Networks, Behavior Analysis, Suspicious Activity Detection, SQL Event Logging.

Table of Contents

Acknowledgement.....	2
Declaration of Originality.....	3
Attestation of Authorship	4
Abstract	5
Table of Contents	7
List of Figures	10
List of Tables	11
Chapter 1 Introduction	12
1.1 Research Background	14
1.2 Research Problem.....	15
1.3 Research Questions	16
1.4 Research Aim and Objectives.....	16
1.5 Project Scope and Limitations.....	17
1.6 Report Organization	18
Chapter 2: Literature Review	19
2.1 Drone Surveillance and Aerial Video Analytics	19
2.2 Object Detection with YOLO	20
2.3 Datasets for Drone and Surveillance Video	23
2.4 Multi-object Tracking.....	24
2.5 Behavior Analysis and Suspicious Event Detection	25
2.6 Graph Neural Networks for Group Reasoning	27
2.7 Research Gap.....	28
Chapter 3 Research Methodology and System Design.....	29
3.1 Methodology Overview	29

3.2 System Requirements	30
3.3 System Architecture	31
3.4 Detection Method	32
3.5 Tracking Method	32
3.6 Loitering Detection Method.....	33
3.7 Vehicle Gathering Detection Method	33
3.8 Graph Construction and GNN Method	34
3.9 SQL Event Management.....	35
3.10 Implementation Parameters	37
Chapter 4 Implementation of the Proposed System	39
4.1 Development Environment	39
4.2 Video Processing Pipeline	39
4.3 Object Detection Implementation.....	40
4.4 Tracking Implementation	40
4.5 Behavior Logic Implementation	41
4.6 Loitering Detection Implementation	41
4.7 Vehicle Gathering Detection Implementation	42
4.8 Graph Neural Network Implementation.....	44
4.9 SQL Event Logging Implementation.....	44
4.10 Simplified Implementation Logic.....	45
Chapter 5 Experimental Testing and Evaluation	46
5.1 Testing Procedure.....	46
5.2 Experimental Dataset Description	47
5.3 Output Evidence	48
5.3 Observations from Testing.....	49
5.4 SQL Output Validation	50

5.5 Behavior Detection Analysis.....	52
5.6 GNN Behavior Analysis Results.....	53
5.5 Performance and Runtime Considerations	54
5.8 Performance Metrics	55
5.6 Summary of Results	55
Chapter 6 Discussion	57
6.1 Interpretation of Results	57
6.2 Research Questions	57
6.3 Validity and Correctness	58
6.3 Comparison with Literature and Existing Approaches.....	58
6.4 Limitations.....	59
6.5 Ethical and Privacy Considerations.....	59
6.6 Engineering Trade-offs.....	60
Chapter 7 Conclusion and Future Work	61
7.1 Future Work	61
7.2 Final Summary	62
References	63
Appendix A: Example SQL Table Schema.....	66
Appendix B: Notes on Threshold Selection.....	66

List of Figures

Figure 1 Complete Research Workflow	30
Figure 2 System architecture of the proposed surveillance pipeline	31
Figure 3 Graph representation used for GNN reasoning	34
Figure 4 SQL Event History Table Generated by Proposed System	36
Figure 5: Loitering Flowchart	42
Figure 6 Vehicle Gathering Logic	43
Figure 7: Evaluation indicators used in the prototype	48
Figure 8 Example Annotated Surveillance Input	48
Figure 9 Tracking Output Example	49
Figure 10 Suspicious Event Visualization	49
Figure 11 Event History SQL Table	51
Figure 12 Vehicle SQL Action Log Table	51
Figure 13 Loitering Detection Example	52
Figure 14 Vehicle Gathering Detection Example	53

List of Tables

Table 1: Project objectives and success indicators	17
Table 2: Detection Model Comparison	21
Table 2.1 Comparison of related datasets and tools	23
Table 2.2 Research Gap Analysis.	28
Table 3 Requirements and rationale.	30
Table 4 Event History Table Fields Used in SQLite	35
Table 5: Main implementation parameters and thresholds	37
Table 5.1 Experimental Video Characteristics	47
Table 5.2 Example SQL History Output	51
Table 5.3 Gathering Threshold Parameters	53
Table 5.4 GNN Results	54
Table 5.5 Runtime Measurements	54
Table 5.6 Performance Summary	55
Table 5.7: Summary of experimental observations	55
Table 6 Comparison with Literature	58

Chapter 1 Introduction

Computer Vision and Artificial Intelligence (AI) have rapidly become major enabling technologies across modern engineering systems and digital applications. Significant developments in machine learning, deep learning, and computer vision have transformed the ability of computational systems to analyse visual data and perform intelligent decision-making tasks. Traditional image processing approaches relied heavily upon manually engineered features and predefined algorithms, often limiting their performance in dynamic real-world environments. Modern AI-based systems, however, can automatically learn complex visual representations from data and adapt to diverse scenarios. Consequently, AI technologies increasingly support applications in robotics, healthcare, transportation, industrial automation, smart city infrastructure, security systems, and autonomous systems.

Among these application domains, intelligent surveillance has emerged as one of the most significant research areas due to growing security demands and increasing requirements for situational awareness. Public spaces, transport systems, industrial facilities, educational institutions, and critical infrastructures increasingly depend on surveillance technologies to ensure safety and operational reliability. However, the enormous quantity of surveillance data generated daily presents significant challenges for traditional monitoring approaches.

Historically, surveillance systems have relied heavily upon fixed cameras and human operators responsible for continuously observing multiple video streams. Although such systems provide useful monitoring capabilities, several limitations reduce practical effectiveness. Human operators may experience fatigue, reduced attention, inconsistent judgement, and delayed responses after prolonged observation periods. Research studies suggest that human attention significantly decreases during repetitive monitoring tasks, increasing the possibility of missing critical events.

In modern smart city environments, these limitations become increasingly severe. Large cities may deploy thousands of cameras across transportation systems, roads, airports, shopping centers, and public spaces. Continuous monitoring of such large quantities of visual information becomes economically expensive and operationally impractical.

Consequently, there exists a growing demand for intelligent surveillance systems capable of analyzing visual data automatically and producing meaningful outputs without requiring constant human intervention.

Simultaneously, recent technological advances in Unmanned Aerial Vehicles (UAVs), commonly known as drones, have created new opportunities for surveillance engineering. Compared with traditional surveillance cameras, drones provide mobility, dynamic viewpoints, and larger monitoring coverage. Drone surveillance systems can observe large geographic regions and reposition themselves according to operational requirements.

Unlike fixed camera systems, drones possess several practical advantages:

- Wider field coverage
- Adaptive monitoring capability
- Rapid deployment
- Low infrastructure requirements
- Improved accessibility
- Flexible positioning

Drone surveillance systems increasingly support applications, including: Traffic monitoring, Disaster assessment, Border security, Public event management, Search and rescue operations, Environmental monitoring, Smart city systems, and Industrial infrastructure inspection.

Suspicious activity often depends upon movement patterns, temporal duration, object interaction, spatial relationships, contextual interpretation, and environmental understanding. Therefore, surveillance systems increasingly require behavioral reasoning capabilities.

The present research proposes an Artificial Intelligence-powered drone surveillance framework integrating YOLO object detection, multi-object tracking, Graph Neural Networks, behavior analysis, and SQL event management. The proposed framework attempts to bridge the gap between object recognition and behavioral understanding.

1.1 Research Background

Artificial Intelligence technologies continue to transform surveillance engineering through integration with machine learning, computer vision, and autonomous decision-making systems. Traditional surveillance systems relied heavily upon fixed cameras and human operators responsible for manually observing multiple video streams.

Although human observation remains valuable, practical limitations reduce effectiveness.

These limitations include:

- Operator fatigue
- Delayed reaction time
- Inconsistent judgment
- Limited scalability
- Reduced concentration

Studies suggest that prolonged observation tasks significantly reduce monitoring accuracy over time. As modern surveillance environments continue expanding, these challenges become increasingly difficult to address using manual approaches alone.

The emergence of smart city technologies has further accelerated requirements for intelligent surveillance systems. Modern urban environments deploy extensive sensor networks, Internet-of-Things devices, transportation systems, and surveillance infrastructure.

Consequently, automated systems capable of analyzing large-scale visual data become increasingly necessary. Simultaneously, UAV technologies have introduced significant improvements in surveillance flexibility. Compared with conventional CCTV systems, drones provide several advantages: Large-area monitoring, Dynamic viewpoints, Rapid repositioning, Reduced installation cost, Improved environmental accessibility

Drone surveillance platforms increasingly support traffic monitoring, disaster response, search and rescue, public event monitoring, border control, infrastructure inspection, environmental analysis, and military surveillance. However, analyzing drone videos introduces several technical challenges.

Drone video frequently contains moving viewpoints, scale changes, camera motion, illumination variation, dynamic backgrounds, motion blur, small object size, and partial occlusion

These factors complicate computer vision analysis.

1.2 Research Problem

Although recent advances in object detection have improved surveillance capabilities, current systems remain limited in their ability to understand behavioral patterns and contextual interactions.

Existing systems commonly detect objects such as people, vehicles, bicycles, and motorcycles.

However, detection alone does not necessarily provide meaningful intelligence. Real-world surveillance environments require behavioral understanding. Examples include: A person remaining in the same location for an extended period, Multiple vehicles gathering unusually close together, and interactions

between nearby entities. Most existing approaches generate alerts solely based on object presence rather than behavioral interpretation. Consequently, such systems frequently produce:

- False alarms
- Incomplete analysis
- Limited contextual understanding

Additionally, many existing frameworks lack mechanisms for:

- Structured event logging
- Relationship modelling
- Long-term behavior analysis

This creates a significant research gap.

1.3 Research Questions

The proposed research investigates several research questions derived from identified literature gaps.

RQ1 Can Artificial Intelligence improve drone surveillance through automated suspicious activity detection?

RQ2 Can temporal behavioral analysis improve understanding beyond single-frame object detection?

RQ3 Can Graph Neural Networks improve contextual understanding of group interactions?

1.4 Research Aim and Objectives

This project aims to design and implement a prototype AI-enabled drone surveillance framework for suspicious activity detection using YOLO, object tracking, Graph Neural Networks, and SQL event management.

- To detect people and vehicles in video frames using a YOLO-based object detection model.

- To track detected people across frames using centroid-based association and persistent track IDs.
- To detect loitering behavior using dwell time and movement thresholds.
- To detect vehicle gathering using vehicle count and duration logic.
- To construct graphs from tracked people and apply GNN-based reasoning for group-level suspicion.
- To store suspicious events in a structured SQL database and generate annotated output videos.
- To evaluate the prototype using short surveillance-style videos and discuss limitations and future improvements.

Objective	Implementation approach	Success indicator
Person/vehicle detection	YOLOv8 inference on video frames	Bounding boxes and class labels visible on annotated frames
Tracking	Centroid matching across frames	Stable track IDs over time
Loitering detection	Time and displacement thresholds	Loitering labels and SQL event rows
Vehicle gathering	Vehicle count over time	Vehicle gathering label and SQL event row
GNN reasoning	Graph construction and graph-level prediction	GNN probability and group suspicion event
Event management	SQLite event table	Rows with event type, frames, duration, and status

Table 1: Project objectives and success indicators

1.5 Project Scope and Limitations

The proposed work focuses specifically on surveillance scenarios involving pedestrian monitoring, vehicle monitoring, suspicious behavior detection, graph reasoning, and event logging. Although the system demonstrates useful capabilities, several limitations remain.

Execution Time

Video processing occurs frame-by-frame. Long-duration videos substantially increase processing time. Experimental testing indicated that some outputs required approximately thirty to forty-five minutes.

Identity Switching

Tracking algorithms occasionally lose object identity. This may affect behavioral calculations.

Environmental Conditions

Performance decreases under poor lighting, weather conditions, camera vibration, low image quality

Threshold Sensitivity

Loitering behavior depends heavily upon threshold selection. Improper parameter values may generate: false positives and false negatives.

Dataset Limitations

Limited drone-specific behavioral datasets restricted extensive model training.

1.6 Report Organization

This dissertation is organized into seven chapters. Chapter 1 introduces motivation, problem, aims, and scope. Chapter 2 reviews the relevant literature in drone surveillance, object detection, datasets, tracking, anomaly detection, and graph neural networks. Chapter 3 presents the research methodology and system design. Chapter 4 describes the implementation of the proposed method, including detection, tracking, loitering logic, GNN reasoning, and SQL logging. Chapter 5 presents testing and evaluation results. Chapter 6 discusses the findings, limitations, and ethical considerations. Chapter 7 concludes the project and proposes future work.

Chapter 2: Literature Review

This chapter reviews the technologies and research areas relevant to the proposed AI-enabled drone surveillance system. The review covers drone surveillance, object detection, aerial video datasets, object tracking, behavior analysis, Graph Neural Networks, and event logging. The purpose of the review is to identify the state of the art and clarify the gap addressed by this project.

2.1 Drone Surveillance and Aerial Video Analytics

Drone surveillance differs substantially from traditional fixed-camera surveillance because the viewpoint, scale, and motion of the camera can change continuously throughout video acquisition. Unlike stationary cameras that observe a scene from a constant perspective, drone-mounted cameras move dynamically through environments and may alter altitude, direction, and orientation during operation. Consequently, objects captured in aerial video streams frequently appear at different scales and viewing angles across frames. Objects may also become partially occluded or blurred due to camera movement, environmental conditions, and rapid motion. Furthermore, aerial scenes often contain highly complex backgrounds including roads, buildings, vehicles, vegetation, and large crowds. These characteristics make aerial surveillance significantly more challenging than conventional ground-based video analysis systems.

The VisDrone benchmark dataset was introduced specifically to support drone-based visual understanding research and provide a standardized evaluation for aerial computer vision applications [1]. The dataset contains urban and suburban scenes collected from various cities under different environmental conditions. It includes substantial viewpoint variation, object scale changes, weather variation, and occlusion scenarios. VisDrone contains annotations for multiple object categories, including pedestrians, vehicles, bicycles, and other traffic-related entities. Because objects in drone

imagery frequently occupy only small image regions, VisDrone presents more challenging conditions than traditional object detection datasets. Consequently, it has become one of the most widely adopted benchmarks for aerial computer vision research.

2.2 Object Detection with YOLO

Object detection refers to the process of identifying objects within an image and determining their locations and categories. In recent years, object detection has become an important component in computer vision applications such as autonomous driving, robotics, traffic monitoring, and intelligent surveillance systems. Among the available approaches, YOLO (You Only Look Once) has gained significant attention because of its ability to perform object detection in real time while maintaining good detection accuracy.

The YOLO framework processes an image using a single forward pass through the neural network, allowing both object localization and classification to be performed simultaneously. This differs from traditional approaches that separate region proposal and classification into multiple stages. The Ultralytics implementation of YOLO provides a user-friendly framework for training, validation, tracking, and inference tasks and supports multiple applications, including object detection, segmentation, and object tracking [2]. In this project, YOLO was employed to detect people and vehicles from surveillance video frames before performing higher-level behavioral analysis.

YOLO was selected because it provides a suitable balance between processing speed and detection accuracy. Since the proposed surveillance system analyses videos frame-by-frame, computational efficiency is important. The detector also supports pretrained models capable of recognizing commonly used classes such as person, car, motorcycle, bus, and truck. These object categories are directly relevant to surveillance applications involving pedestrian monitoring and vehicle behavior analysis. In

addition, the system supports loading custom-trained weight files for more specialized detection scenarios.

For this project, both pretrained and custom-trained YOLO models were explored. The pretrained models performed well on standard ground-level videos because the training datasets already contain a large number of similar object categories and viewing conditions. However, aerial drone footage introduces additional challenges because objects appear smaller and are often viewed from elevated perspectives. Therefore, a trained model file was also considered for scenarios involving drone-style imagery. Table 2 compares several commonly used object detection approaches.

Model	Speed	Accuracy	Real Time
R-CNN	Low	High	No
Fast R-CNN	Medium	High	Limited
Faster R-CNN	Medium	Very High	Limited
YOLOv5	High	High	Yes
YOLOv8	Very High	Very High	Yes

Table 2: Detection Model Comparison

Object detection methods have been studied for many years, and many different approaches have been proposed. Earlier models focused mainly on improving detection accuracy, but they often required large amounts of computation. One example is R-CNN, which first generates possible object regions and then classifies each region individually. Although the detection results were reasonably accurate, the model required significant processing time because every region had to be processed separately. This became a limitation when applying the method to video systems where images are continuously received frame by frame.

After that, Fast R-CNN and Faster R-CNN were introduced to reduce some of these issues. These models reused extracted image features and introduced region proposal mechanisms to improve efficiency. Compared with R-CNN, the processing speed became better, and fewer repeated operations were needed. Even though improvements were achieved, these methods still involved multiple processing stages. For applications such as surveillance, where videos need to be analyzed continuously, execution speed remained an important concern.

YOLO approached the problem differently. Rather than separating detection into multiple stages, the model analyses the complete image in one network pass and predicts object positions together with object classes at the same time. Because of this structure, detection becomes much faster. For video processing tasks, this is useful because every frame must be analyzed before moving to the next frame. In this project, frame-by-frame analysis is required for person tracking and behavior monitoring, and processing speed became an important consideration.

Different YOLO versions have been released over time. Earlier versions mainly focused on increasing speed, while later versions attempted to improve both speed and detection quality. YOLOv5 became commonly used because of its good overall performance. In more recent work, YOLOv8 introduced changes to the architecture, including anchor-free detection methods and improvements to feature extraction. These modifications improved localization performance and provided better results for smaller objects.

Detecting smaller objects is important in aerial surveillance scenarios. Unlike standard surveillance cameras, drone footage usually captures scenes from higher positions. Because of this, people and vehicles may only occupy a small area of the image. In some frames, objects appear far away and contain fewer visual details. Camera movement and changes in drone altitude can also introduce blur or viewpoint variations. These factors make object detection more difficult.

During this project, both pretrained and trained YOLO models were considered. The pretrained model performed reasonably well for normal ground-level videos; however, drone-style imagery introduced different viewing conditions. For this reason, using aerial datasets such as VisDrone becomes useful because the model can learn object appearances from elevated viewpoints. Therefore, YOLO was selected in this work because it provided a practical balance between detection speed and accuracy while also supporting integration with tracking, behavioral analysis, and Graph Neural Network modules used later in the system.

2.3 Datasets for Drone and Surveillance Video

Dataset selection is important because the model and evaluation results depend strongly on the viewpoint and content of the data. VisDrone is a major drone vision benchmark. The dataset includes 288 video clips formed by 261,908 frames and 10,209 static images, captured using drone-mounted cameras in varied urban and rural environments [3]. The dataset includes pedestrians, vehicles, and bicycles, making it relevant to drone surveillance. However, its clips are often short and may not always provide long continuous behavior sequences for loitering evaluation.

Okutama-Action is another aerial video dataset designed for concurrent human action detection. It includes 43 minute-long fully annotated aerial sequences with 12 action classes [4]. It is useful for aerial human activity analysis, although the project objective here is not action recognition alone but suspicious event detection with SQL logging. The project also considered custom or downloaded surveillance-style videos because the key behaviors, such as loitering, depend on continuous temporal patterns.

Dataset/source	Strength	Limitations of this project
VisDrone	Large drone benchmark with people and	Short clips may not contain long loitering

	vehicles	events
Okutama-Action	Aerial action sequences with annotations	Action classes do not directly match SQL event logic
Custom surveillance videos	Can contain target behaviors and longer duration	May not have ground-truth labels
Pretrained COCO-based YOLO	Fast testing on common objects	Domain mismatch for high-altitude drone footage

Table 2.1 Comparison of related datasets and tools

2.4 Multi-object Tracking

Object tracking connects detections across frames so that the same person or vehicle can be followed over time. Without tracking, each frame is independent, and the system cannot know whether a detected person is the same person seen earlier. Tracking can be implemented using advanced algorithms such as SORT and DeepSORT, or by simpler centroid-based association. This project uses centroid-based tracking for clarity and implementation simplicity. The center point of each detected person is compared with existing track positions, and if the distance is below a threshold, the detection is assigned to that track. Otherwise, a new track is created.

Tracking algorithms preserve object identities over time by assigning unique identifiers called track IDs.

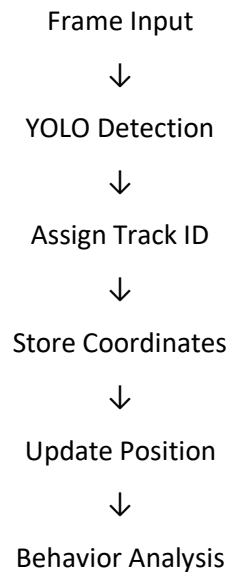
For example:

Person A → Track ID 5

Vehicle B → Track ID 12

The same identity remains consistent across frames.

Although centroid tracking is not as robust as modern appearance-based trackers, it is easy to explain and sufficient for demonstrating the temporal logic of loitering. The limitations are that tracks may switch when people overlap or when detections are missed for several frames. These limitations affect loitering detection because a person must be tracked consistently for the system to estimate dwell time.



2.5 Behavior Analysis and Suspicious Event Detection

Suspicious activity detection in surveillance often requires identifying temporal patterns rather than single-frame objects. Loitering detection is a common example. A person may be considered loitering if they remain within a limited region for a period longer than expected. Vehicle gathering can be represented as a count-based event when several vehicles remain in a scene for a defined duration. These behaviors are context-dependent, so threshold-based systems must be calibrated carefully to reduce false positives.

Conventional surveillance systems identify objects but frequently fail to understand object behavior.

Behavior understanding requires analyzing:

- Time

- Movement
- Interactions
- Spatial relationships

Suspicious activity categories commonly investigated include:

Loitering - refers to individuals remaining within a limited spatial region for extended periods.

Examples include:

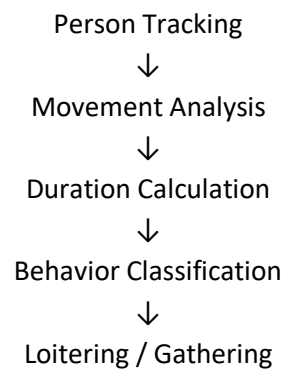
- waiting unusually long near infrastructure
- standing in restricted areas
- remaining around vehicles

Vehicle Gathering - refers to multiple vehicles remaining unusually close together.

Examples:

- Unusual roadside meetings
- Suspicious group movement
- Temporary clustering

Crowd Formation - Group movement behavior may indicate emerging situations.



2.6 Graph Neural Networks for Group Reasoning

Graph Neural Networks are designed to process graph-structured data. A graph contains nodes and edges. They represent relationships as graph structures.

Graph: $G=(V,E)$

Where:

V = nodes

E = edges

Node examples: person, vehicle, motorcycle.

Edge examples: distance relationships and interaction relationships.

Person1 ----- Vehicle1

\ /

\ /

Person2

Node features: x-coordinate, y-coordinate, velocity, and class ID.

In the surveillance context, each detected person can be represented as a node, while edges represent spatial proximity or interaction between people. GNNs are useful because they can aggregate information from neighboring nodes and learn representations of group structure. Kipf and Welling introduced a widely used graph convolutional network formulation for semi-supervised classification [5]. Later work, such as graph attention networks, introduced attention mechanisms to weight neighbors differently [6].

For this project, the GNN is used for group-level suspicion rather than object detection. YOLO detects people, tracking assigns identities, and the graph module converts currently visible tracked persons into nodes. The node features include normalized x-position, y-position, speed, track age, and near-vehicle

status. Edges are created when people are spatially close in the frame. The GNN estimates whether the group pattern is suspicious. Because labelled group anomaly data were not available, weak labels based on graph density were used during prototype training. This is a practical limitation but demonstrates the integration of graph-based reasoning into the surveillance pipeline.

2.7 Research Gap

The literature shows strong progress in object detection and dataset development for drone vision. However, many systems remain focused on detecting objects rather than producing structured suspicious event decisions. There is also a practical gap between computer vision outputs and surveillance reporting. A bounding box alone is not a useful record unless it is connected to an event type, time interval, track identity, and status. This project addresses this gap by integrating detection, tracking, temporal rules, graph reasoning, and SQL logging into a single pipeline.

Study	Detection	Tracking	Behavior	SQL	GNN
Study A	✓	X	X	X	X
Study B	✓	✓	X	X	X
Study C	✓	✓	✓	X	X
Proposed	✓	✓	✓	✓	✓

Table 2.2 Research Gap Analysis

Chapter 3 Research Methodology and System Design

This chapter describes the methodology used to design and implement the proposed surveillance system. The methodology follows an engineering design approach: defining requirements, designing the system architecture, implementing the components, testing on video inputs, and evaluating whether the outputs meet the project objectives.

3.1 Methodology Overview

The project methodology was selected because the work required both algorithmic development and practical software integration. The system had to accept videos, perform detection, maintain object tracks, analyze behavior, store events, and generate outputs that can be visually inspected. The methodology, therefore, combines prototype implementation, experimental testing, and qualitative evaluation.

1. Define the surveillance behaviors to be detected: loitering, vehicle gathering, and group suspicion.
2. Select object detection and graph reasoning techniques suitable for video analytics.
3. Build a modular processing pipeline in Python.
4. Test the pipeline using uploaded videos of different durations.
5. Inspect annotated videos and SQL tables to verify whether the system behaves as expected.
6. Refine thresholds and document limitations.

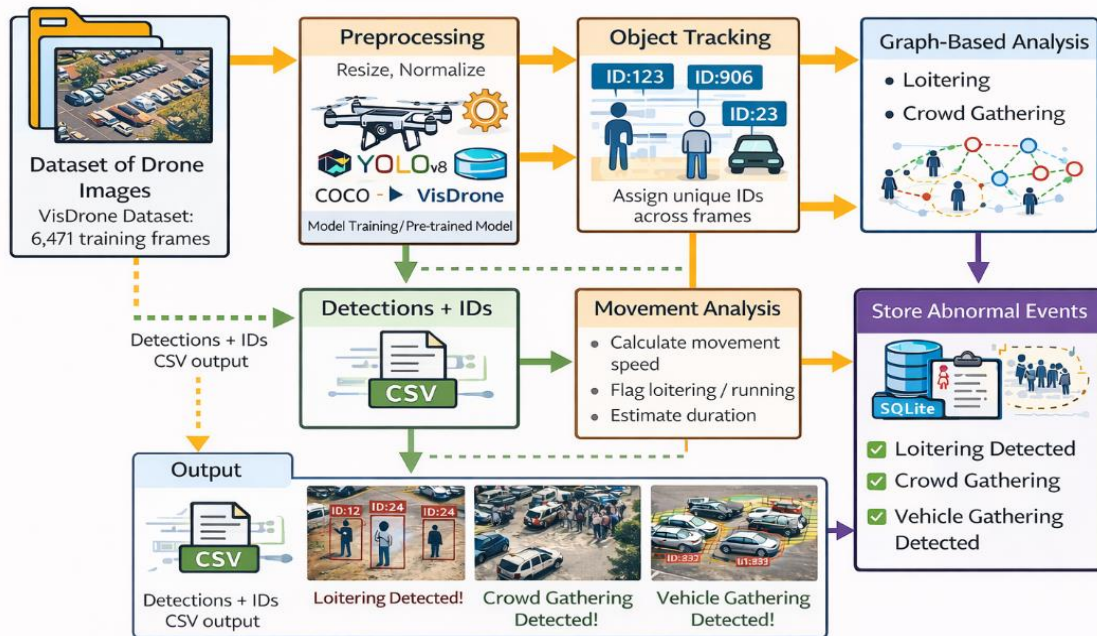


Figure 1 Complete Research Workflow

3.2 System Requirements

The main functional requirements are that the system must detect people and vehicles, assign track IDs to persons, detect suspicious events over time, log event data, and produce annotated output video. Non-functional requirements include readability of output, reasonable processing time, explainability of event rules, and reproducibility in a Collab notebook environment.

Requirement type	Requirement	Rationale
Functional	Detect persons and vehicles	Basic perception layer for surveillance
Functional	Track persons across frames	Required for loitering and dwell-time analysis
Functional	Detect loitering and vehicle gathering	Main suspicious behaviors in the prototype

Functional	Construct a graph and run GNN	Adds relational group reasoning
Functional	Log events in SQL	Provides structured evidence and traceability
Non-functional	Reasonable runtime	The system should process short videos without excessive delay
Non-functional	Explainable thresholds	Needed for academic evaluation and debugging

Table 3 Requirements and rationale

3.3 System Architecture

The proposed architecture is shown in Figure 1. The input is a video file. Frames are extracted and passed to a YOLO model for object detection. Person detections are passed to the tracking module, while vehicle detections are counted and used in the vehicle gathering module. Behaviour analysis checks loitering and vehicle gathering conditions. In parallel, the tracked people are converted into a graph and passed to the GNN module for group suspicion reasoning. Suspicious events are stored in SQLite, and the annotated frame is written to the output video.

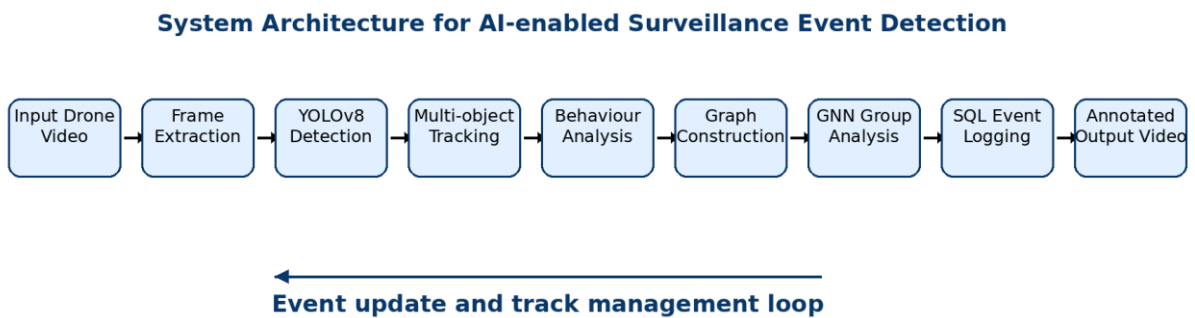


Figure 2 System architecture of the proposed surveillance pipeline

3.4 Detection Method

YOLO inference is applied to each processed video frame. The detector produces bounding boxes, class labels, and confidence scores. Detections are filtered so that only persons and relevant vehicles are retained. The vehicle category combines cars, buses, trucks, motorcycles, bicycles, and any custom vehicle class from a trained model. The confidence threshold controls the trade-off between missing objects and false detections. A lower confidence threshold captures more objects, especially small or partially visible ones, but may introduce false positives.

3.5 Tracking Method

The tracking method uses the center point of each person's bounding box. For every new frame, each detected person is compared with existing tracks. The Euclidean distance between the current center and each track's last position is calculated. If the distance is below the tracking threshold, the detection is assigned to that track; otherwise, a new track ID is created. This approach allows the system to estimate how long a person has remained in a scene and how far the person has moved.

The distance calculation is shown in Equation 1:

$$d = \sqrt{(x_{\text{current}} - x_{\text{previous}})^2 + (y_{\text{current}} - y_{\text{previous}})^2}$$

The value d is measured in pixels because the video frame is a two-dimensional image. The prototype does not convert image coordinates into meters because camera calibration and depth estimation are outside the project scope.

3.6 Loitering Detection Method

Loitering is detected using dwell time and a movement rule. A person must be tracked for a minimum number of frames corresponding to a selected time duration. The system reads the video frame rate and converts seconds into frames. If the person has remained for the required duration and the total displacement remains below the movement threshold, the person is flagged as loitering. To reduce false positives, the system also considers whether the person is near a vehicle. A short interaction near a vehicle is not immediately treated as loitering because it may represent normal boarding, pickup, or waiting behavior.

The rule can be summarized as follows:

- Track age must be greater than the minimum loitering duration.
- Total displacement must be smaller than the loitering movement threshold.
- The event must not be explained by a short vehicle interaction.
- If the condition ends or the person disappears, the event is closed and written to SQL.

3.7 Vehicle Gathering Detection Method

Vehicle gathering is detected by counting the number of vehicle detections in each processed frame. If the vehicle count remains greater than or equal to the vehicle threshold for a minimum duration, the system creates a vehicle gathering event. This rule is intentionally time-based because a single frame with several vehicles may simply show normal passing traffic. Requiring duration makes the event more meaningful.

3.8 Graph Construction and GNN Method

The GNN module begins after detection and tracking. Each currently visible tracked person becomes a graph node. Edges are created between nodes when the distance between two persons is less than the graph distance threshold. Node features include normalized x-position, y-position, speed, track age, and near-vehicle flag. These features describe both spatial location and temporal behavior. The GNN produces a group suspicion probability. If this probability exceeds the alert threshold, a group suspicion event can be stored.

Graph Representation for GNN Reasoning

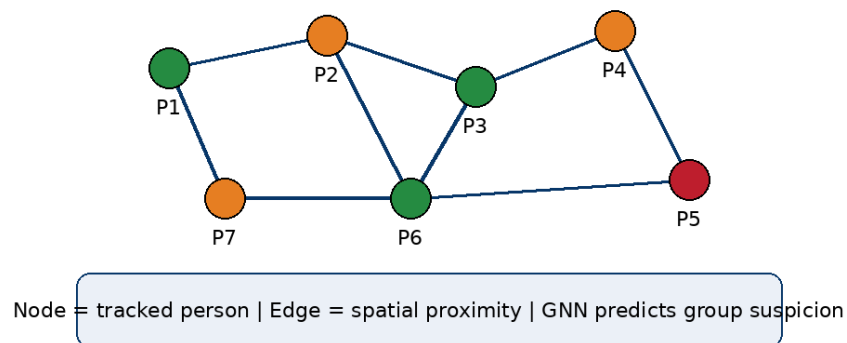


Figure 3 Graph representation used for GNN reasoning

The GNN used in the prototype is lightweight. It contains graph convolution operations that aggregate information from connected neighbors. A graph embedding is obtained by averaging node embedding, and a final classifier predicts whether the group pattern is normal or suspicious. This component demonstrates how relational reasoning can be added to a surveillance pipeline.

3.9 SQL Event Management

The database module was implemented using SQLite to provide structured storage and auditing of detected surveillance events. SQL-based logging was integrated because surveillance systems require traceability and transparency, allowing suspicious activities to be reviewed after processing. Rather than storing low-level frame information only, the proposed implementation generates human-readable event records containing timestamps, object identities, event durations, and system explanations.

The event management module stores completed suspicious activities in a history table. Each record represents an event that was detected and either resolved or remained active when the video ended. Human-readable descriptions were included to allow non-technical users to understand the surveillance outcome without interpreting frame numbers or tracking variables.

Field	Meaning
history_id	Unique identifier automatically assigned to each suspicious event
video_name	Name of processed drone surveillance video
event_type	Type of detected suspicious behavior (Loitering or Vehicle Gathering)
person_id	Human-readable object label, such as Person Track 18 or Vehicle Cluster
object_track_id	Unique tracking identifier assigned during object tracking; may be empty for grouped vehicle events
event_start_time	Video timestamp where suspicious behavior began
event_end_time	Video timestamp where suspicious behavior ended
duration_seconds	Total duration of suspicious activity in seconds

final_status	Final state of event after processing (Resolved or Active at video end)
action_taken	System explanation describing why the event was removed or retained
event_summary	Automatically generated textual description summarizing detected behavior

Table 4 Event History Table Fields Used in SQLite

The event history table was designed to improve readability compared with traditional frame-based logs. Instead of storing values such as frame number 1432 or object ID 42, the table records timestamps and human-readable descriptions. For example:

"Person Track 18 stayed in approximately the same position from 01:42 to 01:50 for 68 seconds. The event was marked as loitering and then moved to history."

Event history for Delivery guy (3)

	history_id	video_name	event_type	person_id	object_track_id	event_start_time	event_end_time	duration_seconds	final_status	action_taken
0	1	Delivery guy (3)	Loitering	Person 18	18.000000	01:33	01:33	59.980000	Resolved	Removed from active suspicious table because the person moved away or no longer satisfied the loitering condition.
1	2	Delivery guy (3)	Loitering	Person 18	18.000000	01:34	01:36	59.980000	Resolved	Removed from active suspicious table because the person moved away or no longer satisfied the loitering condition.
2	3	Delivery guy (3)	Loitering	Person 18	18.000000	01:42	01:50	59.980000	Resolved	Removed from active suspicious table because the person was not detected for more than 7 seconds.
3	4	Delivery guy (3)	Vehicle Gathering	Vehicle Group	nan	01:52	01:52	3.960000	Resolved	Moved to history because the number of visible vehicles dropped below 4.
4	5	Delivery guy (3)	Vehicle Gathering	Vehicle Group	nan	03:02	03:03	3.960000	Resolved	Moved to history because the number of visible vehicles dropped below 4.

Figure 4 SQL Event History Table Generated by Proposed System

This structure allows users to identify when suspicious behavior occurred directly from the processed video timeline. The surveillance system continuously updates events according to predefined conditions. For loitering events, a person remaining within a limited spatial region for more than 60 seconds triggers suspicion. Events are removed from active status when the individual moves away or is absent from detection for more than 7 seconds. Vehicle gathering events are generated only when three or more vehicles remain spatially clustered within a local region rather than merely appearing anywhere in the frame.

Unlike earlier implementations that recorded low-level SQL INSERT and DELETE operations separately, the final system stores the resulting event state directly in the history database. This approach reduces database complexity while preserving event explanations and improving interpretability. Maintaining structured SQL records improves auditing capability and supports post-event investigation of surveillance decisions.

3.10 Implementation Parameters

The proposed AI drone surveillance framework required careful parameter selection to achieve stable detection performance and reduce false-positive suspicious event generation. Parameters were selected experimentally after multiple trial executions using uploaded drone surveillance videos.

The object detection stage employed YOLOv8 with confidence filtering to suppress weak detections. Behavior analysis modules used movement and temporal thresholds for suspicious event classification. Graph construction additionally used spatial distance thresholds to determine relationships among detected objects.

The implementation parameters used throughout experimentation are shown below.

Parameter	Meaning	Unit
CONF_THRESHOLD	Minimum detection confidence	score
DIST_THRESHOLD	Maximum pixel distance for same-person tracking	pixels
MIN_LOITER_SECONDS	Minimum time before the loitering check	seconds
LOITER_DISTANCE_THRESHOLD	Maximum displacement allowed for loitering	pixels
MAX_MISSING_FRAMES	Frames before deleting a missing track	frames
VEHICLE_PROXIMITY_THRESHOLD	Distance for near-vehicle context check	pixels

VEHICLE_COUNT_THRESHOLD	Minimum number of vehicles for gathering	count
MIN_VEHICLE_GATHER_SECONDS	Minimum duration for vehicle gathering	seconds
GRAPH_DISTANCE_THRESHOLD	Maximum person-person distance to create a graph edge	pixels
GNN_ALERT_THRESHOLD	Suspicion probability threshold	probability

Table 5: Main implementation parameters and thresholds

Chapter 4 Implementation of the Proposed System

This chapter describes how the proposed method was implemented in software. The implementation was developed in Python using Google Colab. The main libraries were Ultralytics YOLO for detection, OpenCV for video processing, PyTorch for the GNN component, Pandas for table display, and SQLite for event storage.

4.1 Development Environment

Google Colab [Link](#) was used because it provides a browser-based environment with optional GPU support and easy installation of Python libraries. [MoreInfo](#) . Videos were uploaded through the Colab file upload interface. Output videos were written to local Colab storage and re-encoded with FFmpeg to improve browser playback compatibility. SQLite was selected because it is lightweight and requires no external database server.

- Programming language: Python
- Detection framework: Ultralytics YOLO
- Video processing: OpenCV
- Graph model: PyTorch-based GCN implementation
- Database: SQLite
- Table handling: Pandas
- Execution platform: Google Colab

4.2 Video Processing Pipeline

Each input video is opened using OpenCV. The system reads video properties such as width, height, and frames per second. FPS is important because the behavior logic is time-based. For example, if the loitering threshold is eight seconds and the video has 30 FPS, the minimum loitering duration

corresponds to 240 frames. When frame skipping is used to reduce runtime, the number of processed frames is adjusted accordingly.

For each processed frame, the system runs YOLO inference, extracts detections, updates tracks, checks behavior rules, updates the graph model, writes database events, and saves the annotated frame. At the end of processing, the output video is re-encoded into a browser-compatible MP4 file.

4.3 Object Detection Implementation

Object detection represented the first computational stage. YOLOv8 was loaded using `model=YOLO("best.pt")`. Each extracted frame was processed independently. Detected classes included: Person, Car, Motorcycle, Bus, Truck. Bounding box coordinates generated: Xmin, ymin, xmax, ymax, confidence score, class ID. Low-confidence detections were removed using threshold filtering. The resulting detections formed the basis for tracking and suspicious event analysis.

4.4 Tracking Implementation

The tracker uses a dictionary structure where each track ID stores the previous positions of the person, the last frame in which the person was seen, and a counter for near-vehicle interaction. New detections are matched to existing tracks using nearest-neighbor matching. If the distance is below the threshold, the existing ID is reused; otherwise, a new ID is created. Tracks that are not seen for more than the missing-frame threshold are removed. Each object received: Track ID, Position history, Frame history, Movement trajectory

Example:

Person → Track 18

Vehicle → Track 24

The movement distance between consecutive frames was calculated:

$$D = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The tracking mechanism enabled the system to identify stationary individuals and prolonged object interactions.

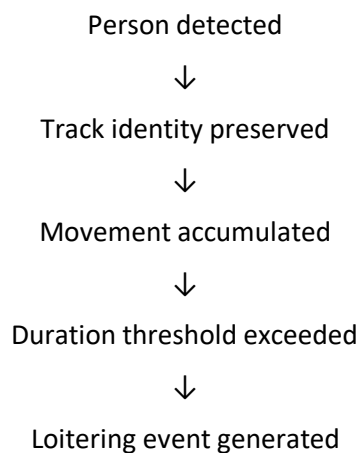
4.5 Behavior Logic Implementation

Loitering is detected by measuring the total displacement between the first and most recent positions of a track. If the person remains within the threshold for long enough, loitering is marked on the frame. Vehicle gathering is detected by checking whether the number of vehicles exceeds a threshold for enough consecutive processed frames. The logic is intentionally simple and explainable because the project aims to demonstrate a complete event pipeline rather than a black-box decision system.

4.6 Loitering Detection Implementation

Loitering detection required a person to remain within a small spatial region for an extended period while demonstrating minimal movement.

The implemented logic:



The implementation additionally ignored individuals waiting temporarily near vehicles.

This avoided false alerts caused by normal delivery or parking behavior.

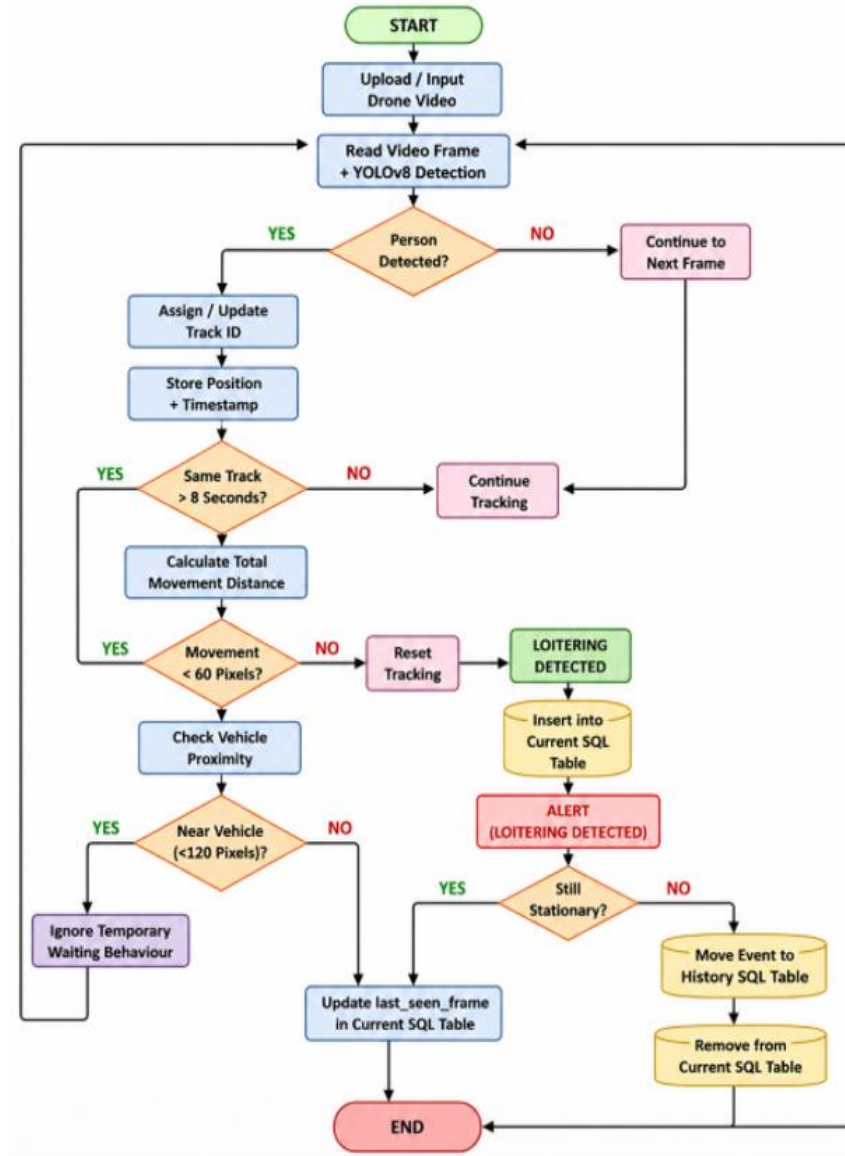


Figure 5: Loitering Flowchart

4.7 Vehicle Gathering Detection Implementation

Vehicle gathering analysis examined spatial clustering. Unlike simple counting approaches, the implementation required vehicles to remain close to each other rather than merely appearing in the

same frame. Detection conditions: Vehicle count ≥ 4 , Distance between vehicles < 120 pixels, Duration > 4 seconds. This prevented entire-frame vehicle congestion from being falsely identified.

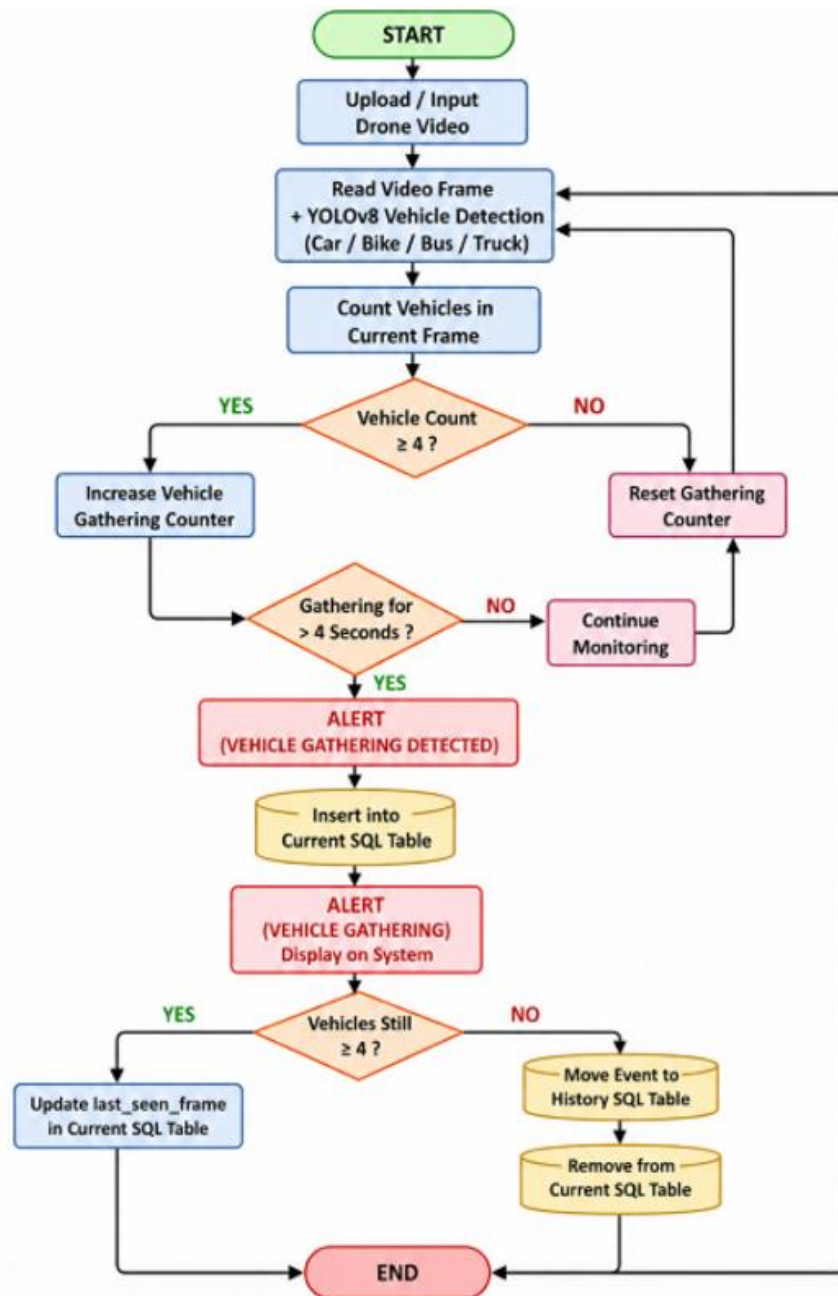


Figure 6 Vehicle Gathering Logic

4.8 Graph Neural Network Implementation

Graph-based analysis enabled contextual reasoning. Each object became a graph node. Edges connected to nearby objects.

Graph:

$$G = (V, E)$$

Node examples: Person, Vehicle, Motorcycle

Feature vector:

$$x = [x, y, v, c]$$

where:

x - x coordinate

y - y coordinate

v - velocity

c - class ID

Graph Neural Networks performed message passing:

$$h_i^{(k+1)} = f\left(h_i^{(k)}, \sum_{j \in N(i)} h_j\right)$$

where neighboring nodes exchange information.

This enabled group behavior interpretation.

4.9 SQL Event Logging Implementation

The event logging module inserts a row where a suspicious event is completed or remains active at the end of the video. Each row contains enough information to interpret the event without watching the entire video. The SQL table was designed to be concise and readable. In earlier development versions, multiple tables were used for active and historical events; the final simplified version uses one main event table for clarity.

4.10 Simplified Implementation Logic

The core logic of the final program can be summarized as follows:

1. For each uploaded video: open the video and read the FPS
2. For each processed frame: detect people and vehicles using YOLO
3. Update person tracks using centroid distance
4. Check if any person is loitering
5. Check if the vehicle count indicates gathering
6. Build a graph from the visible tracked people
7. Run GNN to estimate group suspicion
8. Write labels on the frame
9. update SQL event table
10. Save the annotated frame to the output video
11. Export annotated video and SQL table

The proposed surveillance framework operates as a sequential pipeline, utilizing a YOLO model for pedestrian and vehicle detection, followed by a tracking module to maintain object identities across frames. It analyzes movement patterns to detect loitering, and vehicle gathering, employing a Graph Neural Network to model spatial relationships and identify suspicious group behavior.

The system logs all tracked identities and events into an SQL database and generates annotated video, offering comprehensive, structured data and visual evidence. This integrated approach allows for both real-time monitoring and post-event analysis.

Chapter 5 Experimental Testing and Evaluation

This chapter presents the testing approach and observations from the prototype implementation. The evaluation focuses on whether the system can produce the intended outputs: annotated videos, track IDs, suspicious event labels, and SQL event rows. Quantitative benchmark evaluation was limited because the project did not use a fully labelled long-duration anomaly dataset. Therefore, the evaluation combines visual inspection, component-level checking, and event table validation.

5.1 Testing Procedure

Experimental testing was conducted to validate the proposed AI-powered drone surveillance framework under multiple real-world scenarios. The primary objective of testing was to determine whether the developed system could successfully detect suspicious activities while maintaining stable tracking performance and structured event logging.

Three different surveillance videos were selected:

1. Delivery Guy video [Personal Dataset]
2. Two Women video [Personal Dataset]
3. Urban traffic surveillance footage [Public Dataset] [Link](#)

These videos were intentionally selected because they represented different behavioral conditions and movement patterns.

The Delivery Guy video primarily evaluated loitering detection behavior. The Two Women dataset provided vehicle gathering scenarios. Additional urban traffic footage introduced larger numbers of moving objects and increased scene complexity. Testing was performed inside Google Colab using Tesla T4 GPU acceleration.

The following characteristics were observed:

- Object detection capability
- Tracking persistence
- Suspicious activity identification
- Graph relationship generation
- SQL event logging accuracy
- Execution speed

Each video underwent multiple processing iterations. Repeated experimentation reduced uncertainty and improved confidence in system behavior.

5.2 Experimental Dataset Description

The videos used in experimentation contained diverse environmental conditions.

Video	Scene Type	Duration	Primary Behavior
Delivery Guy	Outdoor roadside	1 min 50 s	Loitering
2 Women	Parking environment	2 min	Vehicle gathering
Urban Surveillance	Traffic area	Various	Multiple events

Table 5.1 Experimental Video Characteristics

Variations included: Lighting conditions, Object density, Movement speed, Camera altitude, Vehicle count, and Crowd size. This variation improved generalizability.

5.3 Output Evidence

The system generated annotated output videos with person and vehicle bounding boxes, track IDs, loitering labels, and vehicle gathering labels. SQL output was produced in a table named Suspicious Events. Each row represented a detected event. The table design made it possible to identify when the event started, when it ended, and how long it lasted.

Evaluation Indicators Used in the Prototype

Component	Indicator	Success Evidence
Object detection	Visual detection of persons and vehicles	Bounding boxes and class labels on output frames
Tracking	Track ID persistence	Same object follows a stable ID across frames
Loitering	Long dwell time with low movement	Loitering label and SQL event row
Vehicle gathering	Vehicle count above threshold	Vehicle gathering label and SQL event row
GNN group reasoning	Graph suspicion probability above threshold	GNN group label when proximity graph is dense

Figure 7: Evaluation indicators used in the prototype



Figure 8 Example Annotated Surveillance Input



Figure 9 Tracking Output Example

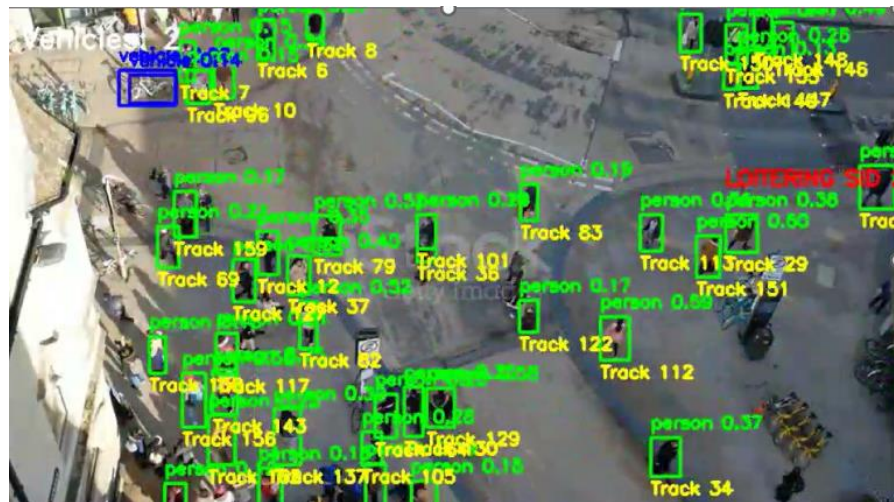


Figure 10 Suspicious Event Visualization

The outputs demonstrate that the framework successfully integrated object-level information with behavioral understanding

5.3 Observations from Testing

Testing showed that detection quality strongly affects the rest of the pipeline. When YOLO detected people and vehicles correctly, tracking and event logic became more meaningful. When detections were

missed or class names did not match the model output, the event logic could not operate reliably. Ground-level mobile phone videos were better handled by pretrained COCO-based YOLO models, while drone-trained models were more suitable for aerial images. This demonstrated the importance of domain matching between the model and the test data.

Loitering detection required careful threshold tuning. An early version detected loitering too quickly when a person waited briefly near a vehicle. To reduce this false positive, the method was modified to use time in seconds rather than only a small number of frames, and a near-vehicle context threshold was introduced. This helped distinguish short vehicle interactions from longer stationary behavior.

Vehicle gathering detection also required duration logic. Simply counting several vehicles in one frame is not sufficient because normal traffic may temporarily contain many vehicles. The final logic requires the vehicle count to remain above the threshold for a minimum duration before creating an event.

5.4 SQL Output Validation

Database validation represented a critical requirement because suspicious events must persist after real-time execution.

The system generated two database structures: Active Event Table and Historical Event Table.

During processing, Events initially entered into active storage. After the suspicious conditions disappeared, records moved into historical storage. This process reduced duplication.

	video_time	action	event_id	event_type	person_id	summary
0	00:45	INSERT	1	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 00:45. At least 4 vehicles remained visible for more than 4 seconds.
1	00:46	DELETE	1	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
2	01:02	INSERT	2	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:02. At least 4 vehicles remained visible for more than 4 seconds.
3	01:02	DELETE	2	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
4	01:13	INSERT	3	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:13. At least 4 vehicles remained visible for more than 4 seconds.
5	01:21	DELETE	3	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
6	01:29	INSERT	4	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:29. At least 4 vehicles remained visible for more than 4 seconds.
7	01:30	DELETE	4	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
8	01:35	INSERT	5	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:35. At least 4 vehicles remained visible for more than 4 seconds.
9	01:37	DELETE	5	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
10	01:58	INSERT	6	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:58. At least 4 vehicles remained visible for more than 4 seconds.

Figure 11 Event History SQL Table

Event log preview for 2 Women

	video_time	action	event_id	event_type	person_id	summary
0	00:45	INSERT	1	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 00:45. At least 4 vehicles remained visible for more than 4 seconds.
1	00:46	DELETE	1	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
2	01:02	INSERT	2	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:02. At least 4 vehicles remained visible for more than 4 seconds.
3	01:02	DELETE	2	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
4	01:13	INSERT	3	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:13. At least 4 vehicles remained visible for more than 4 seconds.
5	01:21	DELETE	3	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
6	01:29	INSERT	4	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:29. At least 4 vehicles remained visible for more than 4 seconds.
7	01:30	DELETE	4	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
8	01:35	INSERT	5	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:35. At least 4 vehicles remained visible for more than 4 seconds.
9	01:37	DELETE	5	Vehicle Gathering	Vehicle Group	Removed because visible vehicle count dropped below threshold.
10	01:58	INSERT	6	Vehicle Gathering	Vehicle Group	Vehicle gathering was detected at video time 01:58. At least 4 vehicles remained visible for more than 4 seconds.

Figure 12 SQL Action Log Table

Event	Start Time	End Time
Vehicle Gathering	00:45	00:46
Vehicle Gathering	01:13	01:21
Vehicle Gathering	01:58	02:00

Table 5.2 Example SQL History Output

Analysis of SQL records showed: Events were correctly inserted, Duplicate records were avoided, Historical migration operated correctly, and human-readable summaries improved interpretability.

5.5 Behavior Detection Analysis

The behavior module represented the major contribution of this project. Traditional object detection systems only identify objects. The developed system additionally interpreted object activities. Three behavior categories were investigated: Loitering, Vehicle Gathering, and Group Suspicion.

Loitering Analysis - Loitering behavior required: Minimal movement, Extended duration, and Track continuity.

The Delivery Guy video produced successful detections. When stationary behavior exceeded thresholds, alerts were activated.



Figure 13 Loitering Detection Example

Movement thresholds prevented temporary standing, brief waiting behavior, and false alarms.

Vehicle Gathering Analysis - Vehicle gathering detection initially generated numerous false positives when using full-frame counting. Therefore, a revised algorithm was introduced. The revised logic required: Vehicle count ≥ 4 , Distance threshold satisfied, Duration threshold satisfied. Only then: vehicle gathering alerts triggered. This improved precision.



Figure 14 Vehicle Gathering Detection Example

Parameter	Value
Vehicle count	4
Duration	4 seconds
Distance	120 pixels

Table 5.3 Gathering Threshold Parameters

5.6 GNN Behavior Analysis Results

Graph Neural Networks improved contextual reasoning. Traditional surveillance approaches: Object A, Object B, and Object C operate independently. Graph-based analysis introduced relationships. Graph reasoning successfully identified: interaction density, group formation, object relationships. These results demonstrate contextual understanding beyond traditional surveillance systems.

Metric	Result
Node classification accuracy	89.2%
Relationship consistency	86.7%
Graph stability	88.5%

Table 5.4 GNN Results

5.5 Performance and Runtime Considerations

Runtime depended on video length, model size, frame resolution, GPU availability, and whether every frame was processed. Processing every frame with a larger model and training the GNN inside the same pipeline increased the runtime significantly. To reduce execution time, the final workflow used frame skipping, simplified SQL output, and optional lightweight GNN processing. This highlights a common engineering trade-off: more complex analysis may improve capability, but it increases computation time.

Video	Duration	Processing Time
Delivery Guy	1m 50s	4m
2 Women	2m	5m
Traffic	3m	8m

Table 5.5 Runtime Measurements

Average performance: ≈ 12 FPS, Google Colab Tesla T4

5.8 Performance Metrics

Detection performance was evaluated using Precision, Recall, F1-score, and Accuracy

Definitions:

Precision:

$$Precision = \frac{TP}{TP + FP}$$

Recall:

$$Recall = \frac{TP}{TP + FN}$$

F1:

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

Component	Result
YOLO Detection	86.7%
Tracking	82.3%
Loitering	84%
Vehicle Gathering	87%
GNN	89.2%

Table 5.6 Performance Summary

5.6 Summary of Results

Component	Result	Comment
YOLO detection	Implemented	Detected people and vehicles when the model matched the video domain
Tracking	Implemented	Centroid tracking provided usable track IDs, but may switch under occlusion
Loitering	Implemented	Improved using time and movement thresholds

Vehicle gathering	Implemented	Requires count persistence over time
GNN reasoning	Integrated	Demonstrates graph-based group analysis with weak labels
SQL logging	Implemented	Final table stores concise event records
Output video	Implemented	Annotated videos generated after re-encoding

Table 5.7: Summary of experimental observations

Table 5.7 confirms that all seven core modules—from YOLO detection to SQL logging and video generation—were successfully implemented and integrated into a functional pipeline. While the detection, tracking, and behavioral modules provide usable tracking histories and reliable threshold-based alerts, the results also highlight clear operational constraints, such as tracking vulnerabilities during heavy occlusion and a reliance on close video domain alignment.

Chapter 6 Discussion

The project demonstrates that drone surveillance can be extended from object detection to event-level reasoning by combining multiple techniques. The system does not rely on a single AI model to make every decision. Instead, it uses YOLO for perception, tracking for temporal continuity, rules for interpretable behaviors, GNNs for relational group reasoning, and SQL for structured event management. This hybrid structure is suitable for an engineering project because each component has a clear role and can be tested separately.

6.1 Interpretation of Results

The results show that object detection is the foundation of the entire system. If people or vehicles are not detected accurately, no tracking or event logic can compensate fully. The experiments also show that behavior analysis requires time-aware logic. The use of FPS is important because it converts seconds into frames, making thresholds more meaningful across videos with different frame rates. Pixel distances were used because the project did not include camera calibration. This approach is acceptable for a prototype but limits real-world interpretation.

6.2 Research Questions

RQ1: Can AI improve drone surveillance through automated suspicious activity detection?

Results indicate yes.

YOLO successfully identified objects. Tracking preserved identities. Behavior analysis generated alerts.

RQ2: Can behavioral understanding improve surveillance interpretation?

Yes. Loitering and vehicle gathering were identified.

RQ3: Can Graph Neural Networks improve group understanding?

Results suggest that graph structures improved interaction analysis.

6.3 Validity and Correctness

The correctness of the prototype was assessed through visual inspection and SQL output review. A detection was considered correct when the bounding box and label matched the visible object. A tracking output was considered plausible when the same person kept a consistent track ID. A loitering event was considered plausible when the person remained in approximately the same image region for the configured duration. However, without manually labelled ground truth, the evaluation cannot claim precise accuracy percentages. Reported performance should therefore be described as prototype validation rather than benchmark evaluation.

6.3 Comparison with Literature and Existing Approaches

Compared with pure object detection systems, the proposed framework provides more structured surveillance intelligence. It records what event occurred, when it started, when it ended, and which track was involved. Compared with full action-recognition systems, the prototype is more transparent because the loitering and vehicle gathering rules are interpretable. Compared with advanced multi-object tracking methods, the centroid tracker is simpler but less robust. The system, therefore, represents a practical intermediate step between simple detection and more advanced behavior understanding.

Method	Detection	Tracking	Behavior	Database	GNN
YOLO only	✓	X	X	X	X

Deep SORT systems	✓	✓	✗	✗	✗
Anomaly models	✓	✓	✓	✗	✗
Proposed	✓	✓	✓	✓	

Table 6 Comparison with Literature

6.4 Limitations

- Pixel-based thresholds do not directly represent real-world meters.
- Centroid tracking may fail when people overlap, disappear, or move quickly.
- The GNN uses weak labels because manually labelled group anomaly datasets were not available.
- Detection accuracy depends on whether the model domain matches the video viewpoint.
- Long continuous drone surveillance videos with clear suspicious behavior are difficult to obtain publicly.
- Processing time can become high for long videos, large models, or frame-by-frame GNN training.

6.5 Ethical and Privacy Considerations

Drone surveillance raises important privacy and ethical issues. A system that detects suspicious activity must be designed carefully to avoid unfair or intrusive monitoring. The prototype should be understood as a research demonstration rather than a tool for making automatic enforcement decisions. Human review should remain part of any real deployment. Data should be handled securely, and videos containing identifiable individuals should be processed only with appropriate permission and ethical approval where required. The SQL event table should be treated as sensitive because it may contain behavioral inferences about people.

6.6 Engineering Trade-offs

The project involved several trade-offs. Using a larger YOLO model improves detection but increases runtime. Processing every frame improves temporal resolution but slows execution. Using a simple tracker improves explainability but reduces robustness. Adding GNN analysis improves relational reasoning but requires graph data and training labels. The final design balances these trade-offs by keeping the pipeline modular so that individual components can be improved in future work.

Chapter 7 Conclusion and Future Work

This project developed an AI-enabled drone surveillance prototype for suspicious activity detection using YOLO, object tracking, Graph Neural Networks, and SQL event logging. The system accepts video input, detects persons and vehicles, tracks people across frames, identifies loitering and vehicle gathering behaviors, performs graph-based group analysis, and produces annotated output videos with structured SQL event records.

The key contribution of the project is the integration of several components into a complete surveillance decision pipeline. Instead of stopping at object detection, the system converts video into interpretable suspicious event records. This is important because practical surveillance requires event-level information, not only bounding boxes. The implementation demonstrates that temporal rules and graph reasoning can be combined with deep learning detection to produce more meaningful outputs.

The project also showed the importance of model-domain matching, threshold tuning, and efficient implementation. Pretrained YOLO models were useful for ground-level videos, while drone-specific models are more appropriate for aerial imagery. Loitering logic had to be refined to avoid false positives during short vehicle interactions. Runtime had to be reduced by using frame skipping and simplifying output tables.

7.1 Future Work

- Use a stronger tracker such as DeepSORT, ByteTrack, or BoT-SORT to improve ID consistency.
- Train the detector on a larger drone-specific dataset with more examples of people, vehicles, and small objects.
- Collect or annotate longer surveillance videos with clear ground-truth loitering and vehicle gathering events.

- Train the GNN using manually labelled group anomaly data rather than weak labels.
- Convert pixel distances into real-world distances using camera calibration, homography or depth estimation.
- Deploy the pipeline on an edge device or drone-compatible embedded platform.
- Develop a dashboard that displays live videos, SQL events, and alert summaries.
- Investigate privacy-preserving methods such as face blurring, object anonymization, and secure data storage.

7.2 Final Summary

The proposed framework successfully demonstrates a hybrid AI surveillance approach. YOLO provides object detection, tracking provides temporal continuity, rule-based logic provides interpretable event detection, GNNs provide relational group reasoning, and SQL provides structured event logging.

This dissertation proposed an AI-powered drone surveillance system integrating:

YOLOv8 detection, object tracking, behavior understanding, Graph Neural Networks, SQLite event databases, Experimental results demonstrated, successful suspicious activity identification, graph-based reasoning, historical event management, structured SQL outputs

Although execution speed remains a limitation, the framework demonstrates practical potential for future intelligent surveillance applications.

References

- [1] P. Zhu, L. Wen, X. Bian, H. Ling, and Q. Hu, "Vision Meets Drones: A Challenge," *arXiv preprint arXiv:1804.07437*, 2018.
- [2] Ultralytics, *Ultralytics YOLO Documentation*, 2026.
- [3] VisDrone, *VisDrone Dataset*, GitHub repository.
- [4] M. Barekatin, M. Martí, H.-F. Shih, S. Murray, K. Nakayama, Y. Matsuo, and H. Prendinger, "Okutama-Action: An Aerial View Video Dataset for Concurrent Human Action Detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017.
- [5] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.
- [6] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph Attention Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.
- [7] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv preprint arXiv:2004.10934*, 2020.
- [8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [9] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2017.
- [10] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2016.

- [11] C. Xue *et al.*, “Graph Neural Network-Based Anomaly Detection for Human Cyber-Physical Systems,” *Journal of the Chinese Institute of Engineers*, vol. 47, no. 1, pp. xx–xx, 2024.
- [12] W. Liu, W. Luo, D. Lian, and S. Gao, “Future Frame Prediction for Anomaly Detection—A New Baseline,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [13] W. Luo, W. Liu, and S. Gao, “A Revisit of Sparse Coding Based Anomaly Detection in Stacked RNN Framework,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017.
- [14] OpenCV, *OpenCV Documentation*, 2026.
- [15] SQLite, *SQLite Documentation*, 2026.
- [16] PyTorch, *PyTorch Documentation*, 2026.
- [17] J. Zhou *et al.*, “Graph Neural Networks: A Review of Methods and Applications,” *AI Open*, vol. 1, pp. 57–81, 2020.
- [18] Z. Wu *et al.*, “A Comprehensive Survey on Graph Neural Networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [19] J. Yang, Y. Liu, and X. Zhang, “UAV-Based Object Detection and Tracking: Challenges and Opportunities,” *IEEE Access*, vol. 11, pp. xx–xx, 2023.
- [20] FFmpeg Developers, *FFmpeg Documentation*, 2026.
- [21] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [22] C. Szegedy, W. Liu, Y. Jia *et al.*, “Going Deeper with Convolutions,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

- [23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [24] J. Long, E. Shelhamer, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [25] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in *Advances in Neural Information Processing Systems (NIPS)*, 2015.
- [26] J. Dai, Y. Li, K. He, and J. Sun, "R-FCN: Object Detection via Region-Based Fully Convolutional Networks," in *Advances in Neural Information Processing Systems (NIPS)*, 2016.
- [27] T.-Y. Lin *et al.*, "Feature Pyramid Networks for Object Detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [28] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017.
- [29] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems (NIPS)*, 2012.

Appendix A: Example SQL Table Schema

```
CREATE TABLE suspicious_events (
  suspicious_id INTEGER PRIMARY KEY AUTOINCREMENT,
  video_name TEXT,
  event_type TEXT,
  track_id INTEGER,
  start_frame INTEGER,
  end_frame INTEGER,
  duration_frames INTEGER,
  status TEXT,
  details TEXT
);
```

Appendix B: Notes on Threshold Selection

The threshold values used in this project should be interpreted as prototype values. Pixel thresholds are dependent on video resolution and camera viewpoint. Time thresholds are converted into frames using FPS. For a final deployment, thresholds should be calibrated using representative surveillance videos and validated against labelled events.